
Practice Exercise 03: ElGamal — Nonce Reuse Attack

Context

We work in the multiplicative group $\mathbb{Z}_p^*$.

- Public parameters: prime p , generator g
- Alice owns the secret key a , and publishes $g^a \bmod p$
- Bob encrypts a plaintext x to Alice by picking exponent b (nonce) and sending:

$$(g^b, y) = (g^b, x \cdot g^{ab}) \pmod{p}$$

If Bob accidentally **reuses the same exponent** b for multiple messages, then all those ciphertexts have the **same** g^b . That also means they have the **same mask** g^{ab} . This breaks confidentiality.

Your Task

You will:

1. Detect nonces that were reused ($O(N)$)
2. Use ONE known plaintext to recover all the other plaintexts in the reuse group

Required functions

You must implement **two** functions:

```
def detect_reuse(ciphertexts):  
    """  
    input:  list of ( $g^b$ ,  $y$ ) pairs  
    output: dict mapping each  $g^b$  to list of indices of ciphertexts that share that  $g^b$   
            must run in  $O(N)$   
    """  
  
def recover_with_known_plain(group_indices, known_index, ciphertexts, x_known, p):  
    """  
    input:  
        group_indices: indices that share the same  $g^b$  (nonce reuse group)  
        known_index:   one index in that group for which plaintext  $x_{\text{known}}$  is known  
        ciphertexts:   list of ( $g^b$ ,  $y$ ) pairs  
        x_known:       known plaintext  
        p:             modulus  
  
    output:  
        dict: index  $\rightarrow$  recovered plaintext for all indices in the group  
    """
```

Implementation Idea

Suppose ciphertexts i and j share the same g^b (same nonce). If you know a plaintext x_1 :

$$g^{ab} \equiv y_1 \cdot x_1^{-1} \pmod{p}$$

Then every other plaintext in that same group is:

$$x_i \equiv y_i \cdot (g^{ab})^{-1} \pmod{p}$$

Dataset

You are given:

System parameters

$p = 95552571855512758838931826791459251019769786805418331004116987637225501521453$

$g = 2$

$g^a = 30830235116662899284371888138332536123855106370370527317876741729102828038282 \pmod{p}$

Ciphertexts (indexed 0..7), each entry is (g^b, y) :

0:

$g^b = 61078443944907540135559149192518503315098864116780697835784992182229203100944$
 $y = 38922467505518474804448446761511814375029180030267443666344772743527104310635$

1:

$g^b = 94027116721640382560628901552325318322340406357085853531014337257321040276214$
 $y = 83672438047882773416302643979814814719892762945915539393325554355223431015057$

2:

$g^b = \text{same as index 1}$
 $y = 36996698308698519852950607205684006956693722239972008639302184484580393692085$

3:

$g^b = \text{same as index 1}$
 $y = 34838938810801520758174491006846279889506248592091881677456438500519105251633$

4:

$g^b = \text{same as index 1}$
 $y = 35013762552715112813296642254598686540240360520740723533849051769108022396264$

5:

$g^b = 61078443944907540135559149192518503315098864116780697835784992182229203100944$
 $y = 38922467505518474804448446761511814375029180030267443666344772743527104310635$

6:

$g^ab = 31582246140190397042889825940781403162886294643931483639505918117179207554036$

$y = 48642035245166085663672810883666533146635477335283141688082508969322891629011$

7:

$g^ab = 12753568787377827777069626163612201165222606771373551616795740342836030919950$

$y = 8154202338300051424566705827673382752034428560600667817962752577194793250903$

Known plaintext you must use:

plaintext for index 1 is known:

$x_1 = 210315799752932871275385655158625349661$

Expected recovered plaintexts (to verify your code):

$x_2 = 11414817998352517241632511775950081130644924184707708450948131510278612237527$

$x_3 = 37647135417842058436058856581393711404732674393379078241248765292980707272523$

$x_4 = 83255510639352351944296738375247527864920310643882061203234015571496556961717$

Deliverables

A .py file that contains:

- your two functions
- code calling them
- printing out the recovered results

and Members.txt of your group.